RUNBOOK

How it fits together
Tools & costs
0 · Plan & collect
1 · Build locally
2 · Sanity CMS
3 · GitHub
4 · Cloudflare
5 · Automations
6 · Forms
Everyday workflow
7 · Hand over & launch
Lessons learned
Command cheat sheet

REUSABLE RUNBOOK · BASED ON THE FREEDOM CHURCH DENTON BUILD, OCT 2026

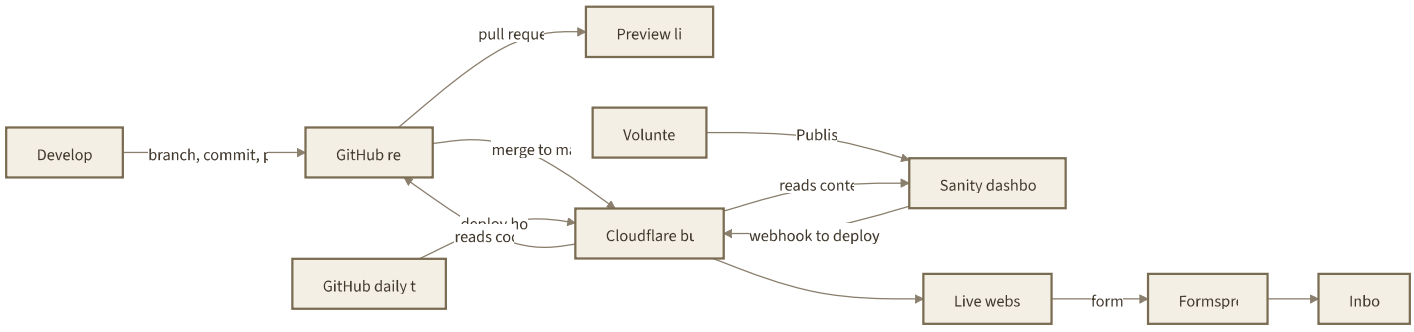
Website Launch Playbook

The exact workflow we used to take a site from an old builder to a fast, free, volunteer-editable website, with code reviewed through pull requests and content published from a dashboard. Follow it in order for your next project.

`Astro + TypeScript` `Sanity CMS` `GitHub` `Cloudflare Workers` `Formspree`

How it fits together

Two separate paths feed one build. Code goes through GitHub; content goes through Sanity. Cloudflare combines them every time it builds.



Path	Who	Trigger	Touches GitHub?
Code change	Developer	Merge a pull request into `main`	Yes
Content change	Volunteer	Click Publish in the dashboard	No
Daily refresh	Nobody	Scheduled GitHub task each morning	Runs there, changes nothing

WHY STATIC

The site is built ahead of time into plain HTML. Visitors never hit Sanity directly, so pages are fast, the site stays up even if Sanity is down, and the free API limits are barely touched. The trade-off: a change appears a few minutes after Publish, not instantly.

Tools & costs

Every service below has a free plan that covers a small organization. Limits were checked in October 2026; check each pricing page again before relying on them.

Tool	Job	Free plan, Oct 2026
Astro + TypeScript	Builds the site into static pages	Free, open source
Sanity	Content storage + editing dashboard (Studio)	20 users, 10k documents, 100 GB assets, 100 GB bandwidth/mo, 250k API requests/mo, 2 webhooks, free Studio hosting. Roles: Administrator and Viewer only.
GitHub	Code, history, pull requests, scheduled tasks	Unlimited private repos. Branch protection needs a public repo or a paid plan.
Cloudflare Workers	Hosting, automatic builds, preview links	3,000 build minutes/mo, 1 build at a time, 20-minute build timeout
Formspree	Delivers contact forms to email	50 submissions/mo, 30-day archive, no CSV export

Domain registrar	Owns the web address	Existing yearly renewal

REJECTED: NETLIFY

In Oct 2026 its free plan gave 300 credits a month at 15 credits per production deploy (about 20 publishes), and it pauses every site on the account at zero. Too tight when every volunteer Publish is a deploy.

NEVER ADD A CREDIT CARD

Sanity starts new projects on a 30-day trial of a paid plan. With no card on file it drops back to Free on its own and cannot bill you; free projects hit hard caps instead of overage charges.

PHASE 0 Plan & collect

Before writing code, gather everything real. Never invent facts for the client.

1. **Inventory the old site.** Download every page and list text, images, links, forms, embeds and old URLs. Old URLs become redirects later.
2. **Find where things live.** Check DNS and records: who hosts the site, who registers the domain, who runs DNS, and where email goes (MX records). Don't touch email records at launch.
3. **Keep real photos only.** Drop stock photos of strangers. List the photos still needed.
4. **Decide the content model.** Which things repeat (events, posts, team members, services) and which pages are one-offs (homepage, about, settings). Each becomes a Sanity type.
5. **Start a facts checklist.** Anything unverified (hours, prices, policies) goes on a list for the client, not onto the site.

USEFUL COMMANDS

Hosting: `nslookup -type=CNAME www.example.org` · DNS host: `nslookup -type=NS example.org` · Email: `nslookup -type=MX example.org`

PHASE 1

Build locally

Build the whole site on your computer first, with a built-in copy of the content so it works before any accounts exist.

Project shape

Copy

```
my-site/
├── src/
│   ├── data/seed.ts        built-in content (fallback when Sanity isn't set up)
│   ├── lib/content.ts      one loader: Sanity if configured, else seed.ts
│   ├── lib/sanity.ts       Sanity client + image helpers (build time only)
│   ├── components/         header, footer, photo, forms…
│   ├── pages/              one file per page
│   └── styles/global.css   design tokens: colors, type, spacing
├── public/_redirects       old URL → new URL
├── studio/                 Sanity Studio (dashboard) + content schemas
├── scripts/export-seed.ts  turns seed.ts into a Sanity import file
├── wrangler.jsonc          Cloudflare settings
└── .github/workflows/      daily rebuild task
```

Steps

1. Create the project with Node 22+: `npm init`, then `npm install astro @sanity/client @sanity/image-url astro-portabletext @astrojs/sitemap sharp`.
2. Write design tokens once in `global.css` and check text contrast meets WCAG AA (4.5:1).
3. Put every piece of real content in `seed.ts`. Pages read only through `content.ts`, never from Sanity or the seed directly.
4. Add `public/_redirects` for every old URL: `/old-page /new-page 301`.
5. Run `npm run dev` (local preview at `localhost:4321`), `npm run build` and `npm run check` until clean.

6 Test at phone, tablet and desktop widths. Test forms, menus and video players by actually clicking them.

`.gitignore`

```
node_modules/
dist/
.astro/
.env
.env.*
!.env.example
studio/node_modules/
studio/dist/
studio/.sanity/
sanity-seed/
```

Copy

PHASE 2 Sanity CMS

Create the project, load the content, and put the dashboard online.

1 Sign up at sanity.io/login with the owner email. Onboarding answers: **Marketing site** · **Astro + TypeScript** · existing site **Yes** · migrating **No**. Ignore the setup commands it shows.

2 Copy the **Project ID** from the project page.

3 Create the Studio in `studio/` : `npm install sanity react react-dom styled-components @sanity/icons` . Import icons per file (`@sanity/icons/Calendar`), because the bundled index export failed to build.

4 Write schemas with plain-English field descriptions, required alt text on images, and singletons for one-off pages (no create/delete actions).

5 Create both `.env` files (never committed):

`.env (site) and studio/.env`

```
SANITY_PROJECT_ID=your-project-id
SANITY_DATASET=production

SANITY_STUDIO_PROJECT_ID=your-project-id
SANITY_STUDIO_DATASET=production
```

Copy

6 Sign in and check the dataset is **public** (read-only; the site reads without a token):

```
cd studio
npx sanity login --provider google
npx sanity dataset list
npx sanity dataset visibility get production
```

Copy

7 Load the content and photos, then preview the dashboard:

```
npm run seed:export
cd studio
npx sanity dataset import ../sanity-seed/seed.ndjson production
npx sanity dev --port 3333
```

Copy

8 Put the dashboard online: `npx sanity deploy` → `https://<name>.sanity.studio`. Then save its app ID so later deploys don't ask:

studio/sanity.cli.ts

```
deployment: {
  appId: 'your-app-id', // shown under Studios in sanity.io/manage
},
```

Copy

9 Invite volunteers under **Members** as **Administrator**. Each uses their own Gmail.

ROLES ON THE FREE PLAN

Only Administrator and Viewer (read-only) exist. Editor is paid; people invited as Editor during the trial become Viewers when it ends. Invite trusted people as Administrator.

PHASE 3 GitHub

Back up the code and set up the branch → pull request → merge workflow.

1 Sign in to the owner GitHub account. In **Settings** → **Emails**, turn on **Keep my email addresses private** and copy the `…@users.noreply.github.com` address.

2 Set the commit identity for **this project only**, so a work email never appears:

```
git config user.name "YourGitHubName"
git config user.email "12345678+YourGitHubName@users.noreply.github.com"
```

Copy

3 Create the repo at github.com/new: **Private**, and leave README, .gitignore and license **off**. Files GitHub adds would clash with your existing history.

4 First commit and push. Put the username in the URL when this computer also has another GitHub account signed in:

```
git init
git branch -M main
git add -A
git status          # check no .env or node_modules
git commit -m "First version of the site"
git remote add origin https://YourGitHubName@github.com/YourGitHubName/my-site.gi
git push -u origin main
```

Copy

RULE

Never commit straight to `main`. Every change goes through a branch and a pull request, even when you are the only developer.

PHASE 4 Cloudflare Workers

Cloudflare recommends Workers for new projects; Pages is labeled legacy. Add the settings file through a pull request first, then connect.

`wrangler.jsonc — "name" must match the Cloudflare project name`

```
{
  "name": "my-site",
  "compatibility_date": "2026-10-01",
  "assets": {
    "directory": "./dist",
    "not_found_handling": "404-page"
  },
  // Required for preview links on branches and pull requests.
```

Copy

```
  "previews": {}
}
```

1. Sign up at dash.cloudflare.com. Skip "add a domain" for now.
2. **Compute** → **Workers & Pages** → **Create** → **Connect GitHub**. Grant access to **only this repository**, pick it, and name the project the same as `"name"` in `wrangler.jsonc`.
3. Build settings: build command `npm run build`, deploy command `npx wrangler deploy`, root `/`, production branch `main`.
4. **Settings** → **Builds** → **Variables and secrets**, on **both** the **Production** and **Previews Base** tabs:

Name	Value
`SANITY_PROJECT_ID`	your project ID
`SANITY_DATASET`	`production`
`PUBLIC_FORMSPREE_CONTACT_ID`	added in phase 6
`PUBLIC_FORMSPREE_PRAYER_ID`	added in phase 6

   Click **Save** in the black bar, then trigger a new build. Saving settings does not rebuild.
5. Check the live address `https://<project>.<account>.workers.dev`: pages load, unknown URLs show your 404 page, old URLs redirect.

PREVIEW LINKS

Every pushed branch other than `main` gets `https://<branch>-<project>.<account>.workers.dev`, posted by a bot on the pull request. Anyone with the link can open it.

PHASE 5 Automations

Two deploy hooks make the site rebuild without anyone touching Cloudflare. Treat hook URLs like passwords.

A · Publish in Sanity rebuilds the site

1. Cloudflare: **Settings** → **Builds (Production)** → **Deploy Hooks** → **Add**. Name `Sanity publish`, branch `main`. Copy the URL.
2. Sanity: sanity.io/manage → project → **API** → **Webhooks** → **Create**. URL = the hook · dataset `production` · trigger on Create, Update, Delete · method POST · filter and projection empty · drafts **off**.
3. Test: publish a change and watch **Deployments** show "Sanity publish - deploy hook".

B · Daily rebuild

Removes finished events, picks up new videos and keeps the copyright year current, even when nobody publishes.

1. Create a second hook named `Daily rebuild`. A separate hook can be replaced alone if it leaks.
2. GitHub repo → **Settings** → **Secrets and variables** → **Actions** → **New repository secret**: `CLOUDFLARE_DEPLOY_HOOK` = the hook URL.
3. Commit the workflow below, then test it with **Actions** → **Daily rebuild** → **Run workflow**.

`.github/workflows/daily-rebuild.yml`

Copy

```
name: Daily rebuild
on:
  schedule:
    - cron: '0 11 * * *' # 6:00 AM Central (daylight time)
  workflow_dispatch:
jobs:
  rebuild:
    runs-on: ubuntu-latest
    steps:
      - name: Trigger Cloudflare build
        env:
          HOOK: ${{ secrets.CLOUDFLARE_DEPLOY_HOOK }}
```

```
        run: |
          if [ -z "$HOOK" ]; then echo "Hook not set; skipping."; exit 0; fi
          curl -fsS -X POST "$HOOK"
```

PHASE 6

Forms

Formspree delivers form messages to an inbox. Form IDs are public by design, so they are safe in page source.

1. Sign up at formspree.io and verify the email. The "Send emails to" list stays empty until you do.
2. Create one form per purpose (e.g. contact, requests). Each ID is the part after `formspree.io/f/`.
3. Add the IDs as `PUBLIC_…` build variables in Cloudflare (Production + Previews Base) and in your local `.env`, then rebuild.
4. Submit each form on the live site and confirm the email arrives. Check spam the first time.
5. To deliver to another inbox later: **Account** → **Linked Emails** → add and verify → choose it on each form.

PUBLIC_ PREFIX

In Astro, only variables starting with `PUBLIC_` may appear in the page. Everything else stays inside the build, so real secrets can't leak by accident.

Everyday workflow

After setup, this is all anyone does.

Content change (volunteers)

Open the dashboard → edit → **Publish**. The site updates in a few minutes. No GitHub, no code.

Code change (developers)

```
git switch main
git pull                                # start from the latest
git switch -c short-task-name           # one branch per task
# …edit files…
git status
git --no-pager diff                     # review your own change
git add <files>
git commit -m "Verb what it does"
git push -u origin short-task-name
```

Copy

1. Open the pull request from GitHub's yellow banner. Write **What / Why / How to test**.
2. Wait for the Cloudflare check. If it fails, read the build log, push a fix to the same branch, and it re-runs.
3. Review **Files changed** and the preview link.
4. **Merge** → **Confirm** → **Delete branch**. Cloudflare deploys `main` automatically.
5. Clean up locally:

```
git switch main
git pull
git branch -d short-task-name
git fetch --prune
```

Copy

PHASE 7 Hand over & launch

The client should own every account, with you added as a helper.

Hand over to one role email (e.g. web@ or admin@)

Service	How	Effort
Sanity	Invite as Administrator → make owner → remove yourself if wanted	~10 min
Formspree	Linked Emails → verify → switch each form	~5 min

Cloudflare	Manage Account → Members → invite as Super Administrator	~10 min
GitHub	Repo Settings → Transfer → reconnect the repo in Cloudflare	~20 min

Store passwords in a password manager, keep 2FA backup codes printed, and make sure at least two people can reach the owner inbox.

Launch (only with the client's approval)

1. Finish the facts checklist with the client.
2. Cloudflare → project → **Domains** → add the real domain.
3. At the DNS host, change **only** the website records as Cloudflare instructs. Leave MX (email) records alone.
4. Test the real domain: pages, forms, give/buy buttons, phone layout.
5. Submit `/sitemap-index.xml` in Google Search Console.
6. Only then cancel the old builder. Check first whether it also bills the domain or email.

Lessons learned

Things that tripped us up the first time.

Typing into the Git viewer

Long `git diff` / `git log` output opens a viewer that eats your keystrokes (and can save stray files). Press `q` to leave, or use `git --no-pager diff .`

Pasted screenshots became project files

Pasting an image into the editor saved it inside the repo. Drag screenshots into chat instead, and check `git status` for untracked surprises.

Two GitHub accounts on one computer

Windows remembered the work login. Putting `username@` in the remote URL made Git sign in as the right account.

Name mismatch warning

Cloudflare used its own project name. Keep `"name"` in `wrangler.jsonc` identical to the project name.

Preview builds failed at "Deploying"

The preview command needs a `"previews": {}` block in `wrangler.jsonc`. Building had succeeded; the log's last lines said exactly what was missing.

Variables don't apply everywhere

Production and Previews Base have separate variable lists, and saving variables never rebuilds. Set both, then trigger a build.

Published but nothing changed

A static site only changes when it rebuilds. Until the Sanity webhook existed, content changes waited for a manual build.

Local dashboard kept stopping

`localhost` stops when the computer sleeps. Deploy the Studio online early.

Video dates looked wrong

YouTube's feed gives upload time, so a Sunday livestream showed as Monday. Label feed dates "Posted …".

Merge created an extra commit

GitHub's default merge adds a "Merge pull request #n" commit. That's normal and records when the change went live.

Command cheat sheet

Command	What it does

`git status`	What changed, what's staged, which branch
`git log --oneline`	History, one line per commit
`git branch -a`	All branches, local and GitHub
`git remote -v`	Where pushes go
`git switch -c name`	Create a branch and move to it
`git add file` / `git commit -m "…"`	Stage, then save a snapshot
`git push -u origin name`	Upload a branch the first time (then just `git push`)
`git pull`	Download the latest from GitHub
`git fetch --prune`	Forget branches deleted on GitHub
`npm run dev` / `build` / `check`	Local preview / production build / type checks
`npx sanity dev` / `deploy`	Run the dashboard locally / put it online (inside `studio/`)
`npm run seed:export`	Turn built-in content into a Sanity import file

Written from the Freedom Church Denton build, October 2026. Prices and limits change; recheck each service's pricing page before quoting costs to a client.